

Case Study: Optimization of a Complex SQL Server Stored Procedure

Project Overview

In one of my previous projects, I worked on optimizing a highly complex SQL Server stored procedure used to load the dashboard data in a multi-tenant healthcare application. The stored procedure integrated data from multiple transactional and lookup tables.

The procedure was heavily utilized by the application landing page, processing millions of records daily.

Initial Problem

The stored procedure had severe performance issues:

- Average execution time: 3–4 minutes, specifically for one tenant with large data volume.
- High CPU and memory consumption
- Timeout failures in the application layer

The procedure included:

- Nested procedure calls
 - Multiple subqueries
 - Multiple table variables
 - Non-optimized filtering conditions
-

Root Cause Analysis

I conducted a detailed performance assessment using:

- SQL Server Execution Plans
- Dynamic Management Views (DMVs)
- SET STATISTICS IO/TIME
- Query Store analysis

The investigation revealed several bottlenecks:

1. The nested SP call was identified as the major bottleneck
 2. Subqueries executing row-by-row
 3. Extensive use of table variable leading to increased I/O
 4. Functions (in the filter condition) executing row by row
-

Optimization Approach

I redesigned and optimized the stored procedure using the following techniques:

1. Nested Procedure Optimization – User Authorization Framework

One of the critical performance bottlenecks identified was a nested stored procedure responsible for fetching the authorized list of objects for the logged-in user. The procedure integrated data from multiple authorization-related tables, including:

- Users
- UserRoleAssociations
- UserRoleAuthorizationDetails

The complexity arose due to highly variable authorization configurations stored in the UserRoleAuthorizationDetails table. Since there was no standardized authorization pattern defined across roles, the procedure had to evaluate multiple business scenarios while determining object-level access permissions.

As a result, the existing implementation contained:

- Highly complex JOIN conditions
- Multiple OR-based filtering scenarios
- Repeated conditional logic
- Table variable to store large intermediate result sets
- Excessive table scans and I/O operations

The original implementation attempted to handle all authorization scenarios within a single static query, which caused inefficient execution plans and severe performance degradation as data volume increased.

Optimization Approach - Nested Procedure

To address the issue, I redesigned the authorization retrieval logic using a dynamic SQL-based approach.

Instead of executing one monolithic query covering every possible authorization scenario, I introduced dynamic query generation that constructed optimized SQL statements based on the specific business conditions applicable to the logged-in user and role combination.

The optimization included:

- Building conditional JOINS dynamically based on authorization type
- Splitting complex OR conditions into optimized execution branches
- Eliminating unnecessary table joins for irrelevant scenarios
- Applying early-stage filtering to reduce dataset size
- Replaced large table variables with temporary tables (#TempTables)
- Introducing targeted indexes on frequently filtered authorization columns

Quantifiable Improvements Achieved

Metric	Before Optimization	After Optimization
Execution Time	3–4 minutes	15–30 seconds
Logical Reads	Extremely High	Reduced by ~80%
CPU Utilization	High during peak load	Reduced significantly

Business Impact

The optimization significantly improved application responsiveness for user login and authorization-based object retrieval. It exhibited improved scalability for high-volume tenants and enabled smoother concurrent user access during peak business operations.

2. Query Refactoring

- Replaced nested subqueries with optimized JOINS and derived tables
- Eliminated redundant calculations by preparing mapped data and storing it in table variable
- Eliminated function calls from WHERE clauses. For example, removed conversion of datetime column to date.

3. Temporary Object Optimization

- Replaced large table variables with indexed temporary tables only where necessary
- Eliminated unnecessary table variables

6. Data Reduction Strategy

- Passed the input filter values to the nested stored procedure to apply early filtering techniques to minimize row processing

Tools & Technologies Used

- Microsoft SQL Server
 - SQL Server Management Studio (SSMS)
 - SQL Profiler
 - Query Store
 - Dynamic Management Views (DMVs)
 - Performance recommendations in Azure portal
-

Performance Improvements Achieved

Metric	Before Optimization	After Optimization
Execution Time	3–4 minutes	15–30 seconds
Logical Reads	Extremely High	Reduced by ~80%
CPU Utilization	High during peak load	Reduced significantly

Business Impact

The optimization significantly improved system stability and user experience:

- Application dashboards loaded seamlessly
- Reduced application timeout incidents
- Lowered infrastructure resource consumption
- Increased operational efficiency for business users

This effort also became a reusable optimization framework for other database modules within the application.